

Code-Layout, Readability and Reusability

Date	02 July 2026
Team ID	XXXXXX
Project Name	AI Powered Debt Relief & Financial Recovery Platform
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Standard 4-space Python PEP 8 layout is used across the FastAPI backend routes, and standard prettier formats are applied to React.js UI files.
2	Proper File Structure	Files and folders are logically organized	Yes	Client frontend files (src/components, src/pages) and server backend folders (api/routes, api/models, api/services) are kept cleanly separated.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Descriptive terms like outstanding balance, monthly income, emi amount, and debt stress index are used instead of short, confusing labels.
4	Function / Method Names	Functions are descriptively named	Yes	Explicit function names such as calculate_financial_health(), generate_settlement_letter(), and get_negotiation_history() explain exactly what they execute.
5	Code Comments	Inline and block comments explain logic	Yes	Code comments clearly explain the debt stress mathematical logic and outline how the backend application securely packages parameters for the Gemini API.

6	Modular Design	Code is split into reusable functions/modules	Yes	Core financial calculation algorithms, AI client configuration workflows, and database entry routines are written inside independent code files.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Consolidated database sessions using the SQLAlchemy ORM utility eliminate duplicate connection setups across multiple server routes.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Implements clean try-except blocks to catch potential network drops or invalid entries when connecting to the external Google Gemini API.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Gemini AI Connection Service	Python / Google GenAI SDK	Reused in the letter creation workspace and the interactive dashboard chat guide routine.	High
2	Financial Calculator Engine	Python / NumPy	Reused across initial user verification views, backend metrics endpoints, and summary reports.	High
3	Database Connection Manager	Python / SQLAlchemy ORM	Imported by every router endpoint file that needs to read or write user records or history histories.	High
4	Custom Numeric Form Input Box	JavaScript / React.js	Reused multiple times on the frontend screens for adding outstanding balances, EMIs, and monthly income values.	High
5	Metrics Card Component	JavaScript / React.js	Reused on the core overview board to visualize financial surplus ratios, settlement goals, and stress index numbers.	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	The backend architecture and frontend components match industry guidelines, creating a clean environment for new developers to step in.
Readability	5	Variables are named after actual business metrics (such as income and debts), making the code easy to understand for any financial application developer.
Reusability	4	Isolating the core logic calculations and the Gemini AI packaging module makes it simple to integrate these modules into separate tools later on.

Documentation / Comments	4	Every operational API layout route and financial validation step contains brief header summaries detailing input criteria and output data payloads.
Overall Score	4.5	An exceptionally clean, responsive, and robust codebase that effectively connects a React view screen with a FastAPI runtime.